გაკვეთილი 5 Mod 7 (7.2)

გაკვეთილი 1 — არსებული პროექტის ასლის შექმნა და ლოკალური/დისტანციური რეპოზიტორიები

1. გაკვეთილის მიზანი

ამ გაკვეთილზე მოსწავლეები ისწავლიან, როგორ გადმოიტანონ GitHub-ზე არსებული პროექტი საკუთარ კომპიუტერში და როგორ იმუშაონ მის ლოკალურ ასლთან.

გაკვეთილის ბოლოს მოსწავლეებს შეეძლებათ:

- Local Repository-სა და Remote Repository-ს შორის განსხვავების ახსნა;
- GitHub-ზე არსებული პროექტის საკუთარ კომპიუტერში გადმოტანა;
- `git clone` ბრძანების გამოყენება;
- კლონირებული პროექტის VS Code-ში გახსნა;
- `git status` ბრძანებით პროექტის მდგომარეობის შემოწმება;
- `git log` ბრძანებით ცვლილებების ისტორიის დათვალიერება;
- `git remote -v` ბრძანებით დისტანციურ რეპოზიტორიასთან კავშირის შემოწმება.

2. თეორიული ნაწილი

2.1. რა არის გუნდური პროექტი GitHub-ზე?

პროგრამირების დროს ხშირად გვიწევს ერთ პროექტზე სხვა ადამიანებთან ერთად მუშაობა.

მაგალითად, წარმოვიდგინოთ, რომ სამი მოსწავლე ქმნის სკოლის ვებსაიტს.

- პირველი მოსწავლე მუშაობს მთავარ გვერდზე (`index.html`);
- მეორე მოსწავლე მუშაობს სტილებზე (`style.css`);
- მესამე მოსწავლე ქმნის საკონტაქტო გვერდს (`contact.html`).

ყველა მოსწავლეს სჭირდება პროექტის ფაილებთან წვდომა.

თუ ისინი ფაილებს ერთმანეთს მხოლოდ Messenger-ით ან ელფოსტით გაუგზავნიან, შეიძლება პრობლემები წარმოიშვას: ვიღაცას ძველი ვერსია ჰქონდეს, ფაილები აირიოს ან რომელიმე ცვლილება დაიკარგოს.

ამ პრობლემის გადასაჭრელად ვიყენებთ Git-სა და GitHub-ს.

Git გვეხმარება პროექტის ცვლილებების აღრიცხვასა და მართვაში, ხოლო GitHub საშუალებას გვაძლევს, პროექტი ინტერნეტში შევინახოთ და სხვა ადამიანებთან ერთად

ვიმუშაოთ.

2.2. Local Repository და Remote Repository

პროექტის რეპოზიტორია შეიძლება არსებობდეს ჩვენს კომპიუტერში ან დისტანციურ სერვერზე.

Local Repository (ლოკალური რეპოზიტორია)

ეს არის პროექტის რეპოზიტორია, რომელიც ჩვენს კომპიუტერში ინახება.

მაგალითად:

```
Desktop/
    school-website/
        index.html
        style.css
        script.js
```

ამ ფაილების შეცვლა ინტერნეტის გარეშეც შეგვიძლია.

ლოკალურ Git რეპოზიტორიაში ასევე ინახება ცვლილებების ისტორია.

Remote Repository (დისტანციური რეპოზიტორია)

ეს არის რეპოზიტორია, რომელიც დისტანციურ სერვერზე, მაგალითად GitHub-ზე, ინახება.

მაგალითად:

```
https://github.com/username/school-website
```

დისტანციური რეპოზიტორიის საშუალებით სხვა ადამიანებსაც შეუძლიათ პროექტზე მუშაობა, შესაბამისი წვდომის არსებობის შემთხვევაში.

ლოკალური და დისტანციური რეპოზიტორიების კავშირი

მთავარი განსხვავება: Local Repository არის ჩვენს კომპიუტერში, Remote Repository კი დისტანციურ სერვერზე.

მაგალითად, როდესაც პროექტში რაიმე ცვლილებას შევიტანთ, თავდაპირველად ის მხოლოდ ჩვენს კომპიუტერში იქნება. GitHub-ზე ცვლილების ასატვირთად მოგვიანებით გამოვიყენებთ `git push` ბრძანებას.

2.3. რა არის Git Clone?

`git clone` არის Git-ის ბრძანება, რომლის საშუალებითაც დისტანციური რეპოზიტორიის ასლს ჩვენს კომპიუტერში ვქმნით.

კლონირებისას გადმოგვაქვს არა მხოლოდ პროექტის ფაილები, არამედ Git-ის ისტორია და დისტანციურ რეპოზიტორიასთან დაკავშირებული ინფორმაცია.

სინტაქსი:

```
git clone repository-url
```

მაგალითი:

```
git clone https://github.com/username/school-website.git
```

სადაც:

- `git` — Git პროგრამას მიმართავს;
- `clone` — რეპოზიტორიის ასლის შექმნის ბრძანებაა;
- `repository-url` — იმ რეპოზიტორიის მისამართია, რომლის გადმოტანაც გვინდა.

თუ პროექტის სახელია `school-website`, Git ჩვეულებრივ ავტომატურად შექმნის ამავე სახელის საქაღალდეს.

```
school-website/
    index.html
    style.css
    script.js
    .git/
```

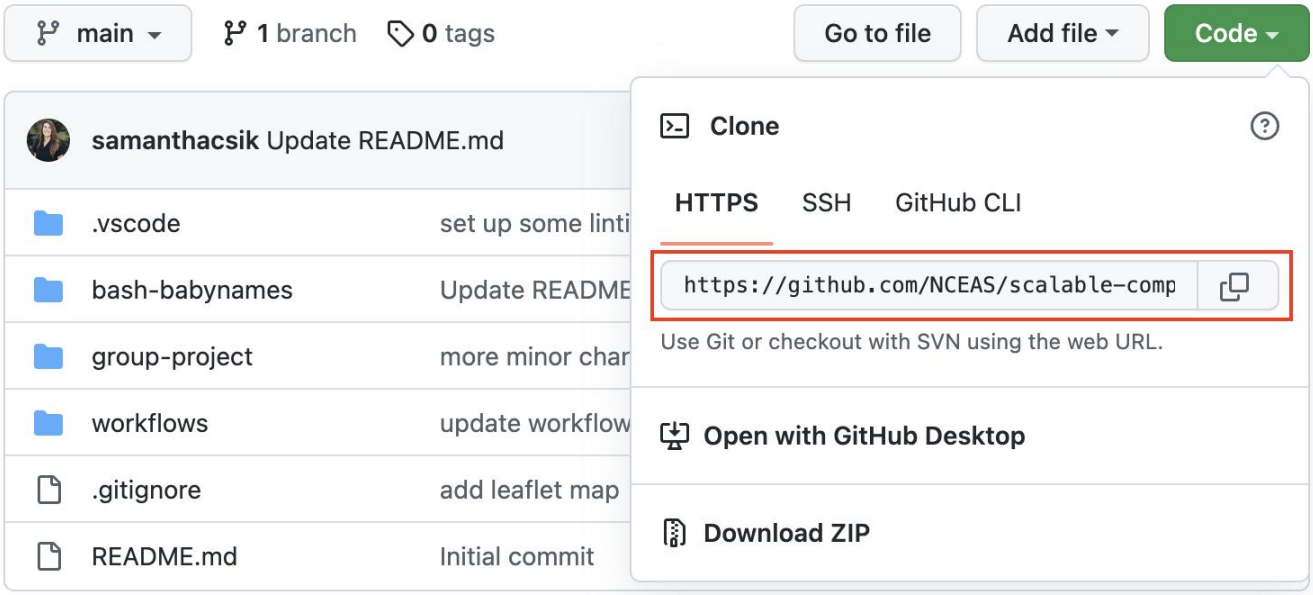
`.git` არის დამალული საქაღალდე, სადაც Git ინახავს რეპოზიტორიის მონაცემებსა და ისტორიას. მისი ხელით წაშლა არ შეიძლება, თუ გვსურს, რომ პროექტი კვლავ Git რეპოზიტორიად დარჩეს.

2.4. როგორ გადმოვიტანოთ პროექტი GitHub-დან?

განვიხილოთ ყველაფერი ეტაპობრივად.

ნაბიჯი 1 — GitHub-ზე პროექტის მოძებნა

ვხსნით GitHub-ს და გადავდივართ სასურველი პროექტის გვერდზე.



learning.nceas.ucsb.edu

ნაბიჯი 2 — პროექტის მისამართის კოპირება

- ვაჭერთ `Code` ღილაკს;
- ვირჩევთ `HTTPS`-ს;
- ვაკოპირებთ რეპოზიტორიის ბმულს.

მაგალითად:

```
https://github.com/username/school-website.git
```

ეს მხოლოდ საილუსტრაციო მისამართია. რეალურ სამუშაოში ვიყენებთ ჩვენი რეპოზიტორიის ბმულს.

ნაბიჯი 3 — Terminal-ის გახსნა

VS Code-ში ვხსნით ტერმინალს:

`Terminal → New Terminal`

ან Windows-ში ვიყენებთ PowerShell-ს / Git Bash-ს.

ვირჩევთ საქაღალდეს, სადაც პროექტის შენახვა გვინდა.

მაგალითად:

```
cd Desktop
```

თუ Windows-ის ტერმინალი სხვა დისკზე ან საქაღალდეშია, საჭირო იქნება შესაბამის მისამართზე გადასვლა.

ნაბიჯი 4 — პროექტის კლონირება

ტერმინალში ვწერთ:

```
git clone https://github.com/username/school-website.git
```

კლონირების წარმატებით დასრულების შემდეგ პროექტი ჩვენს კომპიუტერში გამოჩნდება.

ნაბიჯი 5 — პროექტის საქაღალდეში შესვლა

```
cd school-website
```

ნაბიჯი 6 — პროექტის გახსნა VS Code-ში

```
code .
```

`.` ნიშნავს მიმდინარე საქაღალდეს.

ამ ბრძანებით VS Code-ში გაიხსნება ის პროექტი, რომლის საქაღალდეშიც ვიმყოფებით.

თუ `code .` არ მუშაობს, პროექტის საქაღალდე შეგვიძლია VS Code-ის `File → Open Folder` მენიუდან გავხსნათ.

2.5. Git Status — პროექტის მდგომარეობის შემოწმება

`git status` გვაჩვენებს, რა მდგომარეობაშია ჩვენი პროექტი.

ბრძანება:

```
git status
```

თუ კლონირებულ პროექტში არაფერი შეგვიცვლია, შეიძლება ასეთი შედეგი მივიღოთ:

```
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
```

ეს ნიშნავს, რომ მიმდინარე სამუშაო საქაღალდეში შესანახი ცვლილებები არ გვაქვს და ლოკალური `main` branch Git-ის ბოლო ცნობილი ინფორმაციით ემთხვევა `origin/main`-ს.

ახლა წარმოვიდგინოთ, რომ `index.html` ფაილში შევცვალეთ სათაური:

მანამდე:

```
<h1>Welcome!</h1>
```

ცვლილების შემდეგ:

```
<h1>Welcome to our school!</h1>
```

თუ ცვლილებას შევინახავთ და გავუშვებთ:

```
git status
```

შეიძლება მივიღოთ:

```
Changes not staged for commit:
    modified: index.html
```

ეს ნიშნავს, რომ Git-მა დააფიქსირა ფაილის ცვლილება, თუმცა ის ჯერ commit-ში არ შეგვინახავს.

მნიშვნელოვანია: `git status` არაფერს ცვლის და არაფერს ტვირთავს. ის მხოლოდ გვაჩვენებს პროექტის მდგომარეობას.

2.6. Git Log — ცვლილებების ისტორიის დათვალიერება

Git-ის ერთ-ერთი მთავარი უპირატესობა ცვლილებების ისტორიის შენახვაა.

`git log` ბრძანებით შეგვიძლია ვნახოთ, რა commit-ები შესრულდა პროექტში.

ბრძანება:

```
git log
```

შედეგის მაგალითი:

```
commit a1b2c3d4...
Author: Giorgi <giorgi@example.com>
Date: ...

    Added home page
```

```
commit e5f6g7h8...
Author: Nika <nika@example.com>
Date: ...

    Initial commit
```

ამ მაგალითში მონაცემები პირობითია.

ყოველ commit-ს აქვს:

- უნიკალური იდენტიფიკატორი (hash);
- ავტორი;
- თარიღი;
- აღწერა.

ამრიგად, შეგვიძლია ვნახოთ, ვინ და როდის შეინახა ცვლილებები პროექტის ისტორიაში.

თუ უფრო მოკლე ჩამონათვალი გვინდა, გამოვიყენებთ:

```
git log --oneline
```

მაგალითად:

```
a1b2c3d Added home page
e5f6g7h Initial commit
```

ეს ვარიანტი ბევრად მოსახერხებელია, როდესაც პროექტში ბევრი commit გვაქვს.

2.7. Git Remote — დისტანციური რეპოზიტორიის შემოწმება

როდესაც GitHub-დან პროექტს `git clone` ბრძანებით გადმოვიტანთ, Git ჩვეულებრივ ავტომატურად ქმნის კავშირს დისტანციურ რეპოზიტორიასთან.

ამ კავშირის სანახავად ვიყენებთ:

```
git remote -v
```

მაგალითად, შედეგი შეიძლება იყოს:

```
origin  https://github.com/username/school-website.git (fetch)
origin  https://github.com/username/school-website.git (push)
```

აქ:

`origin` — დისტანციური რეპოზიტორიის სტანდარტული სახელია, რომელსაც Git კლონირებისას ჩვეულებრივ ავტომატურად ანიჭებს.

`fetch` — მიუთითებს მისამართს, საიდანაც Git-ს შეუძლია დისტანციური მონაცემების მიღება.

`push` — მიუთითებს მისამართს, სადაც Git ცვლილებების გაგზავნას შეეცდება, შესაბამისი უფლებების არსებობის შემთხვევაში.

შესაბამისად, `git remote -v` გვეხმარება გავიგოთ, რომელ GitHub რეპოზიტორიასთან არის დაკავშირებული ჩვენი პროექტი.

3. პრაქტიკული მაგალითი — სკოლის ვებსაიტის კლონირება

მასწავლებელი მოსწავლეებს გაუზიარებს წინასწარ შექმნილი GitHub რეპოზიტორიის ბმულს, რომელშიც იქნება შემდეგი ფაილები:

`index.html`

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>School Website</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>
    <h1>Welcome to Our School</h1>
    <p>This is our first team project.</p>
</body>
</html>
```

`style.css`

```
body {
    font-family: Arial, sans-serif;
    background-color: lightblue;
    text-align: center;
}

h1 {
    color: darkblue;
}
```

შემდეგ მოსწავლეები ასრულებენ ბრძანებებს:

```
git clone REPOSITORY_URL
cd school-website

git status
git log --oneline
git remote -v
```

`REPOSITORY_URL` უნდა შეცვალონ მასწავლებლის მიერ გაზიარებული ბმულით.

მოსწავლეები ხსნიან `index.html` ფაილს და ცვლიან:

```
<h1>Welcome to Our School</h1>
```

შემდეგით:

```
<h1>Welcome to Khoni School</h1>
```

ინახავენ ფაილს და ტერმინალში უშვებენ:

```
git status
```

შედეგი: მოსწავლეები დაინახავენ, რომ Git აფიქსირებს `index.html` ფაილის ცვლილებას.

4. პრაქტიკული სავარჯიშოები

სავარჯიშო 1 — პროექტის კლონირება (მარტივი)

დავალება: მოსწავლემ უნდა გადმოიტანოს მასწავლებლის მიერ გაზიარებული GitHub რეპოზიტორია საკუთარ კომპიუტერში.

შესასრულებელი მოქმედებები:

1. გახსენით მასწავლებლის მიერ გაზიარებული GitHub რეპოზიტორია.
2. დააკოპირეთ მისი HTTPS მისამართი.
3. გახსენით VS Code-ის Terminal.
4. გამოიყენეთ `git clone`.
5. გადადით პროექტის საქაღალდეში.
6. გახსენით პროექტი VS Code-ში.

მოსალოდნელი შედეგი: კომპიუტერში უნდა არსებობდეს GitHub-დან დაკლონილი პროექტი და მისი ფაილები.

სავარჯიშო 2 — ფაილების ცვლილებების აღმოჩენა (საშუალო)

დავალება: დაკლონილ პროექტში შეცვალეთ HTML და CSS ფაილები და დააკვირდით, როგორ აფიქსირებს Git ცვლილებებს.

მოთხოვნები:

1. `index.html`-ში შეცვალეთ მთავარი სათაური.
2. დაამატეთ ახალი პარაგრაფი თქვენი სკოლის შესახებ.
3. `style.css`-ში შეცვალეთ ფონის ფერი.
4. შეინახეთ ორივე ფაილი.
5. გამოიყენეთ `git status`.
6. დააკვირდით, რომელ ფაილებს აჩვენებს Git შეცვლილად.

მოსალოდნელი შედეგი:

```
Changes not staged for commit:
    modified: index.html
    modified: style.css
```

მოსწავლემ უნდა ახსნას, რატომ გამოჩნდა ორივე ფაილი შეცვლილად.

სავარჯიშო 3 — Git-ის ისტორიისა და კავშირის შემოწმება (საშუალო)

დავალება: მოსწავლემ უნდა გამოიკვლიოს დაკლონილი პროექტის ისტორია და დისტანციურ რეპოზიტორიასთან კავშირი.

შესასრულებელი ბრძანებები:

```
git log
git log --oneline
git remote -v
```

უპასუხეთ შეკითხვებს:

1. რამდენი commit არსებობს პროექტის ისტორიაში?
2. რა არის ბოლო commit-ის აღწერა?
3. რა განსხვავებაა `git log`-სა და `git log --oneline`-ს შორის?
4. რას ნიშნავს `origin`?
5. რომელი GitHub რეპოზიტორიიდან არის პროექტი დაკლონილი?

მოსალოდნელი შედეგი: მოსწავლეს უნდა შეეძლოს Git-ის ისტორიის წაკითხვა და დაკავშირებული დისტანციური რეპოზიტორიის ამოცნობა.